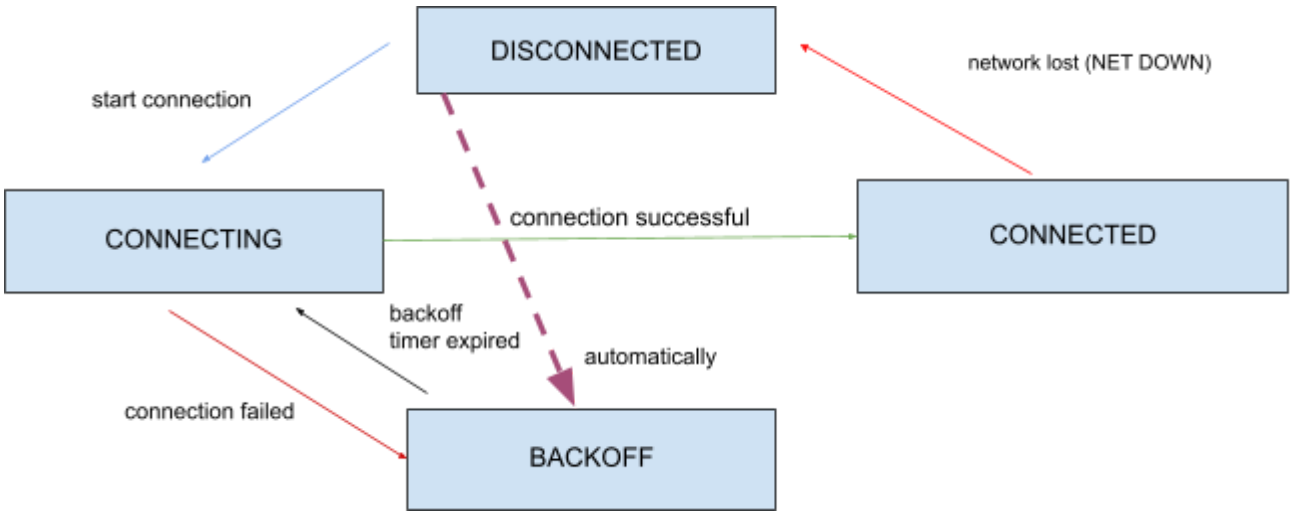
IoT Network / Memory Stability Test Sheet

Webinar 7

State diagram



Network state model

FROM STATE	TRIGGER	TO STATE	LOG
CONNECTED	NET DOWN	DISCONNECTED	20:24:41.501 [NET] state=DISCONNECTED
DISCONNECTED	automatically	BACKOFF	20:24:41.502 [NET] state=BACKOFF, retry=1 backoff_ms=500
BACKOFF	The backoff timer has expired	CONNECTING	20:24:41.964 [NET] state=CONNECTING
CONNECTING	failure	BACKOFF	reconnect=FAILED, state=BACKOFF, retry=2 backoff_ms=50
BACKOFF	The backoff timer has expired	CONNECTING	20:24:42.015 [NET] state=CONNECTING
BACKOFF / CONNECTING (цикл повторюється)	NET UP	CONNECTING	20:24:57.989 [FAULT] network restored, 20:24:57.990 state=CONNECTING

FROM STATE	TRIGGER	TO STATE	LOG
CONNECTING	success	CONNECTED	state=CONNECTED, reconnect=SUCCESS

Expected model: CONNECTED → DISCONNECTED → CONNECTING, and if the connection attempt fails → BACKOFF → CONNECTING.

Actual behavior: After NET DOWN, the device transitions immediately from DISCONNECTED to BACKOFF, bypassing CONNECTING (an unexpected transition not included in the model). It then continuously cycles between BACKOFF and CONNECTING until NET UP.

Risks table

ID	RISK	REQUIREMENT	CONSEQUENCE	HOW TO TEST
RISK-01	The node fails to recover after network loss	R2, R5	Telemetry is not delivered	Scenario A
RISK-02	Messages are not sent after network recovery, and the buffer is not cleared	R3, R5	Buffer overflow; telemetry is not transmitted	Scenario B
RISK-03	The DROP_OLDEST policy does not work during buffer overflow, or dropped messages are not recorded	R4	Silent data loss; the number of lost messages cannot be determined	Scenario C
RISK-04	Duplicate messages occur after reconnection	R6	Double-counting of data and inaccurate analytics	Scenario A-D
RISK-05	Free heap decreases systematically during repeated SEND/STRESS operations (memory leak)	R7	Eventual memory exhaustion, node freeze, or reboot	Scenario D
RISK-06	Reconnect storm: the node attempts	R2	Broker/network overload,	NET DOWN, wait 20 seconds, then

ID	RISK	REQUIREMENT	CONSEQUENCE	HOW TO TEST
	to reconnect too frequently (backoff does not increase)		excessive energy consumption, and CPU usage	run STATUS; inspect reconnect_attempts and backoff_ms in the log
RISK-07	Insufficient evidence: STATUS does not display all required fields	R8	Device behavior cannot be demonstrated or verified, and bugs may go unnoticed	Run STATUS and check whether the following fields are present: state, buffer/capacity, sent, dropped, duplicates, reconnect_attempts, and heap

Network State / Fault Log

Step	Command	State	Buffer	Sent	Dropped	Dup	Reconnects	Heap
1	[Scenario A] RESET STATS + STATUS	CONNECTED	0	0	0	0	0	350868
2	[Scenario A] NET DOWN -> STATUS	BACKOFF	0	0	0	0	240	350868
3	[Scenario A] NET UP -> STATUS	CONNECTED	0	0	0	0	258	350868
4	[Scenario B] RESET STATS -> STATUS	CONNECTED	0	0	0	0	0	350868
5	[Scenario B] STATUS (after NET UP) -> NET DOWN -> SEND 5 times -> NET UP	CONNECTED	0	10	0	5	154	350204
6	[Scenario C] RESET STATS -> NET DOWN -> SEND 12 times -> BUFFER -> STATUS (before NET UP)	BACKOFF	12	0	0	0	223	349908

7	[Scenario D] RESET STATS -> STATUS	CONNECTED	0	0	0	0	0	350868
8	[Scenario D] STRESS 100 ×10 -> STATUS (see Memory Stability Log)	CONNECTED	0	1000	0	0	0	270600
9	[Scenario D] NET DOWN -> STRESS 100 -> STATUS -> NET UP -> STATUS ×5	CONNECTED	0	1000	0	500	357	310600

Execution Log

ID	Commands	Expected	Actual	P/F	Evidence
Scenario A (R2, R5)	RESET STATS -> STATUS -> NET DOWN -> Wait 10-20 sec -> STATUS -> NET UP -> STATUS	Node does not hang and accepts commands. Retry interval grows or stays controlled, no reconnect storm. After NET UP the state becomes CONNECTED and the NET LED turns on.	Node did not hang and accepted commands (STATUS answered, broker=false during NET DOWN). First retry ≈466 ms after DISCONNECTED→BACKOFF (backoff_ms=500), then ≈55 ms between attempts (backoff_ms=50). 240 reconnect attempts in 16.931 s by the STATUS check; 258 attempts over the 21.352 s network outage, until NET UP. After NET UP, the	R2 - Fail R5 - Pass	Log: https://drive.google.com/file/d/14bSmssYBtfZh8jbkDN2VuyH3wN6z_cgM/view?usp=sharing 

			node transitioned to CONNECTED immediately, and reconnect=SUCCESS was logged 1 ms later. The buffer remained empty (buffer=0), and free heap remained unchanged (350868). After NET UP the state becomes CONNECTED and the NET LED turns on.		
Scenario B (R3, R5)	RESET STATS -> NET DOWN -> SEND 5 times -> BUFFER -> NET UP -> STATUS	5 messages queued in the offline buffer (BUFFER shows count=5). After NET UP the node becomes CONNECTED and each of the 5 IDs is published exactly once ([MQTT] publish id=...), the buffer becomes 0, sent=5, duplicates=0	All 5 messages were buffered. After NET UP the node became CONNECTED and the buffer became 0, but every ID was published twice: sent=10 for 5 SEND, duplicates=5. STATUS and the log are consistent (5 IDs × 2 publishes).	R3 - Pass R5 - Pass R6 - Fail	Log: https://drive.google.com/file/d/1s-oVq967Qf6iGTniiNB4LV6fUqiE2bUD/view?usp=sharing 
Scenario C (R4)	RESET STATS -> NET DOWN -> SEND 12 times -> BUFFER -> STATUS -> NET UP	After 12 SEND in offline mode with capacity 8 (DROP_OLDEST): buffer does not exceed 8, dropped=4 and each drop is recorded in the log. After NET UP only the 8 newest IDs are published, once each; the 4 oldest are discarded, with no corrupted or foreign data. Boundary values:	STATUS before NET UP shows buffer=12 (capacity 8). The log shows no drop records and dropped stays 0. flush_start count=12; IDs 5–8 are published twice, IDs 9–12 four times, and the publish order is 9–12, then 5–12. IDs 1–4 are absent from publish while	R4 - Fail R6 - Fail	Log: https://drive.google.com/file/d/1NKKj_i7nSdQjpdFft12FMeNehvYCGRdi/view?usp=sharing  Count 8/8: 

		at 8 messages buffer=8, dropped=0; at 9 messages buffer=8, dropped=1 and the oldest ID is discarded.	dropped=0, so they disappeared without a record. Boundary: with 8 messages the buffer is correct (count=8 capacity=8 dropped=0); at the 9th message the log says overflow policy=DROP_OLDEST dropped_total=0 and queued id=9 count=9/8, so the policy is announced but nothing is dropped and the limit is exceeded.		Count 9/8: 
Scenario D (R7)	RESET STATS -> STATUS -> STRESS 100 -> STATUS -> Repeat step 2 a 10 times	The node remains CONNECTED. After each of the 10 iterations of STRESS 100 + STATUS, the buffer remains empty, dropped and duplicates remain unchanged, and sent increases by 100 per iteration. Free heap shows no systematic irreversible decline.	sent grew by 100 per iteration (up to 1000), buffer, dropped and duplicates stayed 0. Free heap dropped on every iteration by ≈8,000 B (≈80 B per message) and never recovered: 350,868 → 270,600 B (−80,268 B, −22.9%) over 1,000 messages. Iteration 1 was slightly larger (−8,264 B).	R7 - Fail	Log: https://drive.google.com/file/d/1mUCKTWOwrVFRwPIvhB_Pt8Y56-yPL9jL/view?usp=sharing
Scenario D - Stress during NET DOWN (R7, R4, R6)	RESET STATS -> STATUS -> NET DOWN -> STRESS 100 -> STATUS -> NET UP -> STATUS -> Repeat 5 times	In each of the 5 runs (NET DOWN, STRESS 100, STATUS, NET UP, STATUS): before NET UP, the offline buffer contains 8 messages, and the 92 dropped	After NET UP the node was CONNECTED with buffer=0, but in every run sent grew by 200 and duplicates by 100 for 100 generated messages (after	R7 - Fail R4 - Fail R6 - Fail	Log: https://drive.google.com/file/d/1YCibvMAcBolzaJbLoo2dtkHGtX1Pd2nn/view?usp=sharing

		messages are recorded in dropped and the log (R4). After NET UP, only the 8 newest message IDs are published, each exactly once, so sent increases by 8 and duplicates remains unchanged (R5, R6). Free heap shows no systematic irreversible decline (R7).	5 runs: sent=1000, duplicates=500). dropped stayed 0 although 100 messages far exceed capacity 8. Free heap dropped by ≈8,000 B per run (≈80 B per message), the same rate as online, and never recovered: 350,868 → 310,600 B (−40,268 B, ≈80.5 B per message). reconnect_attempts per run: 84, 63, 68, 45, 97 (cumulative 357), depending on how long NET DOWN lasted.		

Duplicates after reconnect. Message ID / Flush Evidence

Scenario B (Step 5): 5 × SEND

ID	Buffered?	Publish count	Expected	P/F
1	YES	2	1	FAIL
2	YES	2	1	FAIL
3	YES	2	1	FAIL
4	YES	2	1	FAIL
5	YES	2	1	FAIL

SEND = 5, sent = 10, duplicated = 5. FAIL

Scenario C (Step 6): 12 × SEND

ID	Buffered?	Publish count	Expected	P/F
1	YES	0	0 (dropped)	OK, but R4 FAIL: drop not recorded
2	YES	0	0 (dropped)	OK, but R4 FAIL: drop not recorded

3	YES	0	0 (dropped)	OK, but R4 FAIL: drop not recorded
4	YES	0	0 (dropped)	OK, but R4 FAIL: drop not recorded
5	YES	2	1	FAIL
6	YES	2	1	FAIL
7	YES	2	1	FAIL
8	YES	2	1	FAIL
9	YES	4	1	FAIL
10	YES	4	1	FAIL
11	YES	4	1	FAIL
12	YES	4	1	FAIL

STATUS before NET UP: sent = 0, duplicates = 0 (no messages have been sent yet, so the counters are zero and cannot confirm or rule out duplicates). However, buffer = 12 while capacity = 8: FAIL (R4). According to the flush log, IDs 5–8 were published twice each, and IDs 9–12 were published four times each: FAIL (R6).

Memory Stability Log

Iteration	Action	Network	Buffer	Sent	Dropped	Duplicates	Free heap	Notes
0	Baseline	CONNECTED	0	0	0	0	350868	baseline after RESET STATS
1	STRESS 100	CONNECTED	0	100	0	0	342604	−8,264 bytes (≈82.6 B/msg); slightly larger, likely one-time init
2	STRESS 100	CONNECTED	0	200	0	0	334604	−8,000 bytes (80 B/msg)
3	STRESS 100	CONNECTED	0	300	0	0	326612	−7,992 bytes (≈79.9 B/msg)
4	STRESS 100	CONNECTED	0	400	0	0	318600	−8,012 bytes

								(≈80.1 B/msg)
5	STRESS 100	CONNECTED	0	500	0	0	310600	−8,000 bytes (80 B/msg)
6	STRESS 100	CONNECTED	0	600	0	0	302600	−8,000 bytes (80 B/msg)
7	STRESS 100	CONNECTED	0	700	0	0	294600	−8,000 bytes (80 B/msg)
8	STRESS 100	CONNECTED	0	800	0	0	286600	−8,000 bytes (80 B/msg)
9	STRESS 100	CONNECTED	0	900	0	0	278600	−8,000 bytes (80 B/msg)
10	STRESS 100	CONNECTED	0	1000	0	0	270600	Total −80,268 bytes, ≈80 B/message, heap does not recover.

Heap decreased by ≈8,000 bytes (≈80 B per message) after every STRESS 100 iteration and never recovered: 350,868 → 270,600 (−80,268 bytes, −22.9%) over 1,000 messages, with buffer=0, dropped=0, duplicates=0. This is a systematic, irreversible decline, which violates R7 (memory leak).

STRESS during NET DOWN

Iteration	Action	Network	Buffer	Sent	Dropped	Duplicates	Free heap	Notes
0	Baseline	CONNECTED	0	0	0	0	350868	baseline after RESET STATS
1	NET DOWN, STRESS 100, NET UP	CONNECTED	0	200	0	100	342612	−8,256 B (≈82.6 B/msg); 100 generated, 200 sent; reconnects: 84

2	NET DOWN, STRESS 100, NET UP	CONNECTED	0	400	0	200	334612	−8,000 B (80 B/msg); +200 sent, +100 dup; reconnects: 147
3	NET DOWN, STRESS 100, NET UP	CONNECTED	0	600	0	300	326612	−8,000 B (80 B/msg); +200 sent, +100 dup; reconnects: 215
4	NET DOWN, STRESS 100, NET UP	CONNECTED	0	800	0	400	318600	−8,012 B (≈80.1 B/msg); +200 sent, +100 dup; reconnects: 260
5	NET DOWN, STRESS 100, NET UP	CONNECTED	0	1000	0	500	310600	−8,000 B (80 B/msg); total −40,268 B (≈80.5 B/msg), heap does not recover; reconnects: 357

STRESS 100 under NET DOWN (5 runs): each run generated 100 messages, but `sent` increased by 200 and `duplicates` by 100, so every message was published twice after NET UP (R6 FAIL). `dropped` stayed 0 although 100 messages exceeded capacity 8 (R4 FAIL). Free heap dropped by ≈8,000 B per run (≈80 B/msg), same as online, and never recovered (R7 FAIL). Reconnect attempts per run varied (45–97) depending on how long NET DOWN lasted.

Bug Report

BUG-01

Title: After NET DOWN reconnect backoff drops from 500 ms to 50 ms and retries keep running every ≈55 ms (reconnect storm)

Requirement/Risk: R2 (retry must use backoff and not create a reconnect storm) / RISK-06

Environment: MacBook Pro, macOS Tahoe 26.5.1, VS Code 1.143.1, PlatformIO IDE 3.3.4 and Wokwi Simulator 4.1.0. ESP32 DevKit v1, homework_buggy firmware (without a version string), PlatformIO Serial Monitor configured with rfc2217://localhost:4000, 115200 baud, monitor_eol = LF, and monitor_filters = time, log2file.

Preconditions: VS Code opened and the Wokwi project launched; after RESET STATS, net=CONNECTED, broker=true, buffer=0

Steps:

1. RESET STATS, STATUS.
2. NET DOWN.
3. Observe [NET] retry=... backoff_ms=... for ≈17 s.
4. STATUS, then NET UP, then STATUS.

Expected: Retry interval grows or at least stays controlled, only a few attempts in 10–20 s, no reconnect storm (R2). Exact values are not specified in R2; growing backoff is the standard behavior.

Actual: The first retry waits ≈466 ms (backoff_ms=500). From retry=2 on, backoff_ms=50 stays constant and the node keeps looping BACKOFF → CONNECTING → BACKOFF about every 55 ms until NET UP. 240 attempts in 16.9 s, 258 by NET UP. DISCONNECTED goes to BACKOFF within 0-2 ms (extra transition compared with the expected model).

IDs/Counters/Timing:

- NET DOWN at 13:12:23.634 → STATUS at 13:12:40.565 (16.931 s) → NET UP at 13:12:44.986 → final STATUS at 13:12:49.235.
- reconnect_attempts: 0 → 240 (net=BACKOFF, broker=false) → 258 (net=CONNECTED).
- Initial delay: 466 ms, from retry=1 / backoff_ms=500 at 13:12:23.636 to the first CONNECTING at 13:12:24.102. From retry=2 (13:12:24.109) to retry=240 (13:12:37.203): 13.094 s / 238 intervals ≈ 55.0 ms per interval. This corresponds to approximately 18.2 attempts/s over that interval and 12.1 attempts/s overall (258 attempts in 21.352 s).
- With an increasing backoff (e.g., 500, 1000, 2000, 4000 ms), substantially fewer attempts would be expected over 16.9 s; the exact count depends on the retry algorithm.
- Timing calculations use the recorded log timestamps.

Heap before/after: 350868 / 350868

Recovery behavior: The node recovered by itself after NET UP: network restored → CONNECTING → CONNECTED → reconnect=SUCCESS within 1 ms, buffer 0. During the observed 21 s the retry loop did not slow down.

Severity: Major: R2 is violated; no workaround is available

Reproducibility: 5/5 Reproducible

Evidence: Log: https://drive.google.com/file/d/14bSmssYBtfZh8jbkDN2VuyH3wN6z_cgM/view?usp=sharing

logs > device-monitor-261026-131150.log

```
13:12:13.694 > [STATS] reset
13:12:18.868 > {"net":"CONNECTED","broker":true,"buffer":0,"capacity":8,"sent":0,"dropped":0,"duplicates":0,"reconnect_attempts":0,"heap":350868}
13:12:23.634 > [NET] state=DISCONNECTED
13:12:23.636 > [NET] state=BACKOFF
13:12:23.636 > [NET] retry=1 backoff_ms=500
13:12:24.102 > [NET] state=CONNECTING
13:12:24.102 > [NET] reconnect=FAILED
13:12:24.109 > [NET] state=BACKOFF
13:12:24.109 > [NET] retry=2 backoff_ms=50
13:12:24.160 > [NET] state=CONNECTING
13:12:24.160 > [NET] reconnect=FAILED
13:12:24.160 > [NET] state=BACKOFF
13:12:24.175 > [NET] retry=3 backoff_ms=50
13:12:24.210 > [NET] state=CONNECTING
13:12:24.218 > [NET] reconnect=FAILED
13:12:24.218 > [NET] state=BACKOFF
13:12:24.219 > [NET] retry=4 backoff_ms=50
13:12:24.269 > [NET] state=CONNECTING
13:12:24.269 > [NET] reconnect=FAILED
13:12:24.269 > [NET] state=BACKOFF
13:12:24.277 > [NET] retry=5 backoff_ms=50
13:12:24.327 > [NET] state=CONNECTING
13:12:24.328 > [NET] reconnect=FAILED
13:12:24.328 > [NET] state=BACKOFF
13:12:24.328 > [NET] retry=6 backoff_ms=50
13:12:24.377 > [NET] state=CONNECTING
13:12:24.377 > [NET] reconnect=FAILED
13:12:24.386 > [NET] state=BACKOFF
13:12:24.386 > [NET] retry=7 backoff_ms=50
13:12:24.436 > [NET] state=CONNECTING
13:12:24.436 > [NET] reconnect=FAILED
13:12:24.436 > [NET] state=BACKOFF
13:12:24.436 > [NET] retry=8 backoff_ms=50
13:12:24.485 > [NET] state=CONNECTING
13:12:24.493 > [NET] reconnect=FAILED
13:12:24.493 > [NET] state=BACKOFF
```

logs > device-monitor-261026-131150.log

```
13:12:37.086 > [NET] retry=238 backoff_ms=50
13:12:37.135 > [NET] state=CONNECTING
13:12:37.144 > [NET] reconnect=FAILED
13:12:37.144 > [NET] state=BACKOFF
13:12:37.144 > [NET] retry=239 backoff_ms=50
13:12:37.194 > [NET] state=CONNECTING
13:12:37.194 > [NET] reconnect=FAILED
13:12:37.194 > [NET] state=BACKOFF
13:12:37.203 > [NET] retry=240 backoff_ms=50
13:12:40.565 > {"net":"BACKOFF","broker":false,"buffer":0,"capacity":8,"sent":0,"dropped":0,"duplicates":0,"reconnect_attempts":240,"heap":350868}
13:12:40.565 > [NET] state=CONNECTING
13:12:40.566 > [NET] reconnect=FAILED
13:12:40.566 > [NET] state=BACKOFF
13:12:40.566 > [NET] retry=241 backoff_ms=50
13:12:40.593 > [NET] state=CONNECTING
13:12:40.593 > [NET] reconnect=FAILED
13:12:40.601 > [NET] state=BACKOFF
13:12:40.601 > [NET] retry=242 backoff_ms=50
13:12:40.652 > [NET] state=CONNECTING
13:12:40.652 > [NET] reconnect=FAILED
13:12:40.652 > [NET] state=BACKOFF
13:12:40.652 > [NET] retry=243 backoff_ms=50
13:12:40.702 > [NET] state=CONNECTING
13:12:40.702 > [NET] reconnect=FAILED
13:12:40.710 > [NET] state=BACKOFF
13:12:40.710 > [NET] retry=244 backoff_ms=50
13:12:40.760 > [NET] state=CONNECTING
13:12:40.760 > [NET] reconnect=FAILED
13:12:40.760 > [NET] state=BACKOFF
13:12:40.768 > [NET] retry=245 backoff_ms=50
13:12:40.810 > [NET] state=CONNECTING
13:12:40.818 > [NET] reconnect=FAILED
13:12:40.819 > [NET] state=BACKOFF
13:12:40.819 > [NET] retry=246 backoff_ms=50
13:12:40.868 > [NET] state=CONNECTING
13:12:40.869 > [NET] reconnect=FAILED
```



BUG-02

Title: Messages are published more than once after reconnect (buffer flush duplicates)

Requirement/Risk: R6 (Duplicates) / RISK-04

Environment: MacBook Pro, macOS Tahoe 26.5.1, VS Code 1.143.1, PlatformIO IDE 3.3.4 and Wokwi Simulator 4.1.0. ESP32 DevKit v1, homework_buggy firmware (without a version string), PlatformIO Serial Monitor configured with rfc2217://localhost:4000, 115200 baud, monitor_eol = LF, and monitor_filters = time, log2file.

Preconditions: VS Code opened and the Wokwi project launched; after RESET STATS, net=CONNECTED, broker=true, buffer=0

Steps:

1. RESET STATS, NET DOWN.
2. SEND 5 times, record IDs.
3. NET UP, STATUS, count [MQTT] publish id=... per ID.

Additional steps:

1. Repeat steps with 12 SEND
2. 5 STRESS 100 during NET DOWN: RESET STATS -> STATUS -> NET DOWN -> STRESS 100 -> STATUS -> NET UP -> STATUS

Expected: Every message ID is successfully published exactly once; duplicates = 0; sent equals the number of generated messages.

Actual:

1. Scenario B: each of IDs 1–5 published 2 times, sent=10 for 5 SEND, duplicates=5.
2. Scenario C: IDs 5–8 published 2 times, IDs 9–12 published 4 times.
3. Scenario D (STRESS 100 during NET DOWN): every run adds 200 to sent and 100 to duplicates (after 5 runs sent=1000, duplicates=500).

IDs/Counters/Timing: Scenario B flush at 19:15:29.628-29.632. Scenario C flush at 19:19:57.511–57.534; each ID is logged twice in a row in [MQTT] publish. Final counters of the stress runs: sent=1000, duplicates=500, reconnect_attempts=357.

Heap before/after: 350868 / 350204 (Scenario B after flush)

Recovery behavior: The node recovered automatically without intervention: after NET UP, it transitioned to CONNECTED (Scenario B: network restored at 19:15:29.613 → CONNECTED at 19:15:29.627, approximately 14 ms; Scenario C: network restored at 19:19:57.497 → CONNECTED at 19:19:57.511, approximately 14 ms) and performed a flush (B: approximately 4 ms, C: approximately 23 ms). flush_end count=0, buffer=0, and commands were accepted.

However, the consequences were not resolved: duplicates had already been published, and the node neither retracted nor deduplicated them. The duplicates counter remained greater than 0 until RESET STATS. Duplication recurred in each of the 5 NET DOWN → NET UP cycles (Scenario D). The heap did not return to its baseline after the flush (350,204 versus 350,868).

Severity: Major: R6 is violated; analytics are distorted

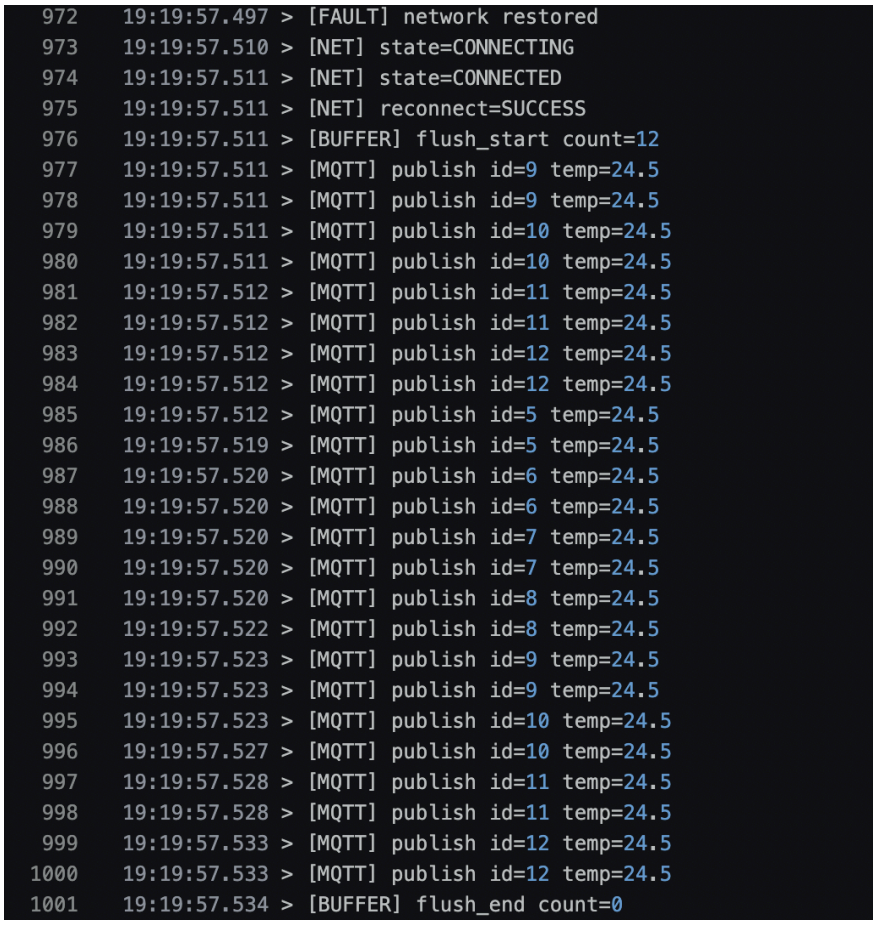
Reproducibility: 5/5 Reproducible

Evidence:
1. SEND x 5
Log: https://drive.google.com/file/d/1s-oVq967Qf6iGTniiNB4LV6fUqiE2bUD/view?usp=sharing



2. SEND x12

Log: https://drive.google.com/file/d/1NKKj_i7nSdQjpdFft12FMeNehvYCGRdi/view



3. 5 STRESS 100 during NET DOWN
Log: https://drive.google.com/file/d/1YCibvMAcBolzaJbLoo2dtkHGtX1Pd2nn/view

```
20:53:06.690 >
{"net":"CONNECTED","broker":true,"buffer":0,"capacity":8,"sent":1000,"dropped":0,"duplicates":500,"reconnect_attempts":357,"heap":310600}
```

BUG-03

Title: Offline buffer count exceeds capacity (12/8); oldest messages lost without dropped/log record

Requirement/Risk: R4 (Buffer boundary, DROP_OLDEST, observable drop) / RISK-03

Environment: MacBook Pro, macOS Tahoe 26.5.1, VS Code 1.143.1, PlatformIO IDE 3.3.4 and Wokwi Simulator 4.1.0. ESP32 DevKit v1, homework_buggy firmware (without a version string), PlatformIO Serial Monitor configured with rfc2217://localhost:4000, 115200 baud, monitor_eol = LF, and monitor_filters = time, log2file.

Preconditions: VS Code opened and the Wokwi project launched; after RESET STATS, net=CONNECTED, broker=true, buffer=0

Steps:

1. RESET STATS, NET DOWN.
2. Execute SEND 12 times (also check the buffer after 8 and 9 SEND commands).
3. BUFFER, STATUS.
4. NET UP, save the flush log.

Expected: buffer ≤ 8; dropped = 4 with a log record for each drop; after NET UP only the 8 newest IDs are published, the 4 oldest are discarded.

Actual:

1. 8 messages: count=8 capacity=8 dropped=0, correct.
2. 9th message: overflow policy=DROP_OLDEST dropped_total=0 and queued id=9 count=9/8. The policy is announced, but nothing is dropped and the limit is exceeded.
3. 12 messages: STATUS shows "buffer":12, dropped stays 0, no drop records in the log.
4. NET UP: flush_start count=12 instead of 8; publish order is 9, 10, 11, 12, then 5…12 (wrong composition and order).

IDs/Counters/Timing:

- 9th message overflow at 19:19:33.785.
- Flush 19:19:57.511 → 19:19:57.534 (flush_start count=12, flush_end count=0).
- Scenario C IDs ≥ 5 are published, IDs 1–4 are absent from publish while dropped=0, so their disappearance is not recorded.

Heap before/after: 350868 / 349908 (Scenario C after 12 SEND, −960 B)

Recovery behavior: The node did not hang and recovered its operational state: network restored at 19:19:57.497 → CONNECTING at 19:19:57.510 → CONNECTED at 19:19:57.511 → flush_start count=12 → flush_end count=0 (19:19:57.534), with buffer=0. IDs 1–4 were lost without being logged (dropped=0 both before and after the flush), while IDs 5–12 were published 2–4 times. The node does not detect this issue or adjust the counter. In subsequent Scenario D runs, the node continued accepting messages and clearing the buffer, meaning that functionality was preserved, but the defect recurred in every cycle.

Severity: Critical: data loss occurs without being recorded anywhere

Reproducibility: 5/5 Reproducible

Evidence: Log: https://drive.google.com/file/d/1NKKj_i7nSdQjpdFft12FMeNehvYCGRdi/view?usp=sharing

```
 972   19:19:57.497 > [FAULT] network restored
 973   19:19:57.510 > [NET] state=CONNECTING
 974   19:19:57.511 > [NET] state=CONNECTED
 975   19:19:57.511 > [NET] reconnect=SUCCESS
 976   19:19:57.511 > [BUFFER] flush_start count=12
 977   19:19:57.511 > [MQTT] publish id=9 temp=24.5
 978   19:19:57.511 > [MQTT] publish id=9 temp=24.5
 979   19:19:57.511 > [MQTT] publish id=10 temp=24.5
 980   19:19:57.511 > [MQTT] publish id=10 temp=24.5
 981   19:19:57.512 > [MQTT] publish id=11 temp=24.5
 982   19:19:57.512 > [MQTT] publish id=11 temp=24.5
 983   19:19:57.512 > [MQTT] publish id=12 temp=24.5
 984   19:19:57.512 > [MQTT] publish id=12 temp=24.5
 985   19:19:57.512 > [MQTT] publish id=5 temp=24.5
 986   19:19:57.519 > [MQTT] publish id=5 temp=24.5
 987   19:19:57.520 > [MQTT] publish id=6 temp=24.5
 988   19:19:57.520 > [MQTT] publish id=6 temp=24.5
 989   19:19:57.520 > [MQTT] publish id=7 temp=24.5
 990   19:19:57.520 > [MQTT] publish id=7 temp=24.5
 991   19:19:57.520 > [MQTT] publish id=8 temp=24.5
 992   19:19:57.522 > [MQTT] publish id=8 temp=24.5
 993   19:19:57.523 > [MQTT] publish id=9 temp=24.5
 994   19:19:57.523 > [MQTT] publish id=9 temp=24.5
 995   19:19:57.523 > [MQTT] publish id=10 temp=24.5
 996   19:19:57.527 > [MQTT] publish id=10 temp=24.5
 997   19:19:57.528 > [MQTT] publish id=11 temp=24.5
 998   19:19:57.528 > [MQTT] publish id=11 temp=24.5
 999   19:19:57.533 > [MQTT] publish id=12 temp=24.5
1000   19:19:57.533 > [MQTT] publish id=12 temp=24.5
1001   19:19:57.534 > [BUFFER] flush_end count=0
```

```
 904   50.059 > [NET] reconnect=FAILED
 905   50.066 > [NET] state=BACKOFF
 906   50.066 > [NET] retry=223 backoff_ms=50
 907   53.258 > {"net":"BACKOFF","broker":false,"buffer":12,"capacity":8,"sent":0,"dropped":0,"duplicates":0,"reconnect_attempts":223,"heap":349908}
 908   53.259 > [NET] state=CONNECTING
```

Count 8/8:

```
 450   19:19:27.608 > [NET] state=BACKOFF
 451   19:19:27.608 > [NET] retry=111 backoff_ms=50
 452   19:19:30.389 > [BUFFER] queued id=8 count=8/8
 453   19:19:30.390 > [NET] state=CONNECTING
```

Count 9/8:

```
 512   19:19:31.067 > [NET] retry=126 backoff_ms=50
 513   19:19:33.785 > [BUFFER] overflow policy=DROP_OLDEST dropped_total=0
 514   19:19:33.785 > [BUFFER] queued id=9 count=9/8
```

BUG-04

Title: Free heap decreases by ≈80 bytes per message and never recovers (memory leak)

Requirement/Risk: R7 (Memory stability) / RISK-05

Environment: MacBook Pro, macOS Tahoe 26.5.1, VS Code 1.143.1, PlatformIO IDE 3.3.4 and Wokwi Simulator 4.1.0. ESP32 DevKit v1, homework_buggy firmware (without a version string), PlatformIO Serial Monitor configured with rfc2217://localhost:4000, 115200 baud, monitor_eol = LF, and monitor_filters = time, log2file.

Preconditions: VS Code opened and the Wokwi project launched; after RESET STATS, net=CONNECTED, broker=true, buffer=0

Steps:

1. RESET STATS, STATUS.
2. STRESS 100, STATUS.
3. Repeat step 2 ten times and record heap after each iteration.
4. Additional run: 5 x (NET DOWN -> STRESS 100 -> STATUS -> NET UP -> STATUS).

Expected: Heap fluctuates within a small band; no systematic, irreversible decline.

Actual: Heap drops by ≈8,000 B on every iteration (≈80 B per message) and is never released. Iteration 1 is slightly larger (−8,264 B). buffer=0, dropped=0, duplicates=0 in the online run, so the leak is not related to buffering.

IDs/Counters/Timing:

- Online run, heap per iteration: 350868 → 342604 → 334604 → 326612 → 318600 → 310600 → 302600 → 294600 → 286600 → 278600 → 270600.
- Total: −80,268 B (−22.9%) for 1,000 messages.
- Rough linear estimate: heap exhausted after about 4,400 messages.
- NET DOWN run (5 iterations): 350868 → 342612 → 334612 → 326612 → 318600 → 310600 (−40,268 B, ≈−8,000 B per run).

Heap before/after: 350868 / 270600 (online, 10 iterations); 350868 / 310600 (with NET DOWN, 5 iterations)

Recovery behavior: The node does not release memory on its own. After each STRESS 100, the heap remains lower (350,868 → 270,600). It does not increase while idle: [STRESS] start heap=270600 matches the heap value reported in the previous STATUS. NET DOWN/NET UP and the flush do not restore memory either (350,868 → 310,600 after 5 cycles).

The node remained CONNECTED and did not hang within the tested range of 1,000 messages (heap ≈77% of its initial value). Behavior under memory exhaustion (estimated at ≈4,400 messages based on a linear projection: reboot, hang, or panic) was not tested. The heap can return to 350,868 in a subsequent run only after restarting the simulator.

Severity: Critical: memory leak

Reproducibility: Reproducible

Evidence:
1. Stress 100 without NET DOWN
Log: https://drive.google.com/file/d/1mUCKTWOwrVFRwPIvhB_Pt8Y56-yPL9jL/view?usp=sharing

```
1034  0.121 > {"net":"CONNECTED","broker":true,"buffer":0,"capacity":8,"sent":1000,"dropped":0,"duplicates":0,"reconnect_attempts":0,"heap":270600}
1035  0.776 > [STRESS] start count=100 heap=270600
1036  0.776 > [MQTT] publish id=1001 temp=24.5
```

2. Stress 100 during NET DOWN

Log: https://drive.google.com/file/d/1YCibvMAcBolzaJbLoo2dtkHGtX1Pd2nn/view?usp=sharing

```
4845  20:53:03.061 > [BUFFER] flush_end count=0
4846  20:53:06.690 > {"net":"CONNECTED","broker":true,"buffer":0,"capacity":8,"sent":1000,"dropped":0,"duplicates":500,"reconnect_attempts":357,"heap":310600}
4847
```

Conclusion

Based on testing the homework_buggy firmware in Wokwi, the IoT node is unstable under non-ideal network conditions: four of the eight requirements are met (R1, R3, R5, R8) and four are violated (R2, R4, R6, R7). The node

does not hang, restores the CONNECTED state within approximately 1–14 ms after NET UP, and performs a buffer flush. However, it duplicates messages, exceeds the buffer capacity, fails to record data loss, and does not release memory.

The most critical product risks are: (1) a potential memory leak of approximately 80 B per message, with exhaustion estimated after approximately 4,400 messages, meaning the node will eventually stop functioning; (2) silent data loss and buffer overflow (potential memory corruption); (3) duplicated telemetry, which distorts analytics; and (4) a reconnect storm that increases the load on the broker and power supply.

What to verify on real hardware: an extended run until heap exhaustion (reboot / watchdog / panic), while monitoring fragmentation; real Wi-Fi and MQTT behavior under declining RSSI, broker restarts, and broker rate limiting; timing measurements using the device clock rather than host timestamps (the simulator has output gaps of 2.7–3.4 s); device reboot during offline operation (the RAM buffer is lost); deduplication on the broker side (MQTT QoS, message IDs); and extended NET DOWN periods exceeding 20 s to determine whether the backoff is capped.

Requirement	**Result**	**Basic**
R1 Network states	PASS	All 4 states were observed in [NET] logs (state diagram + transition table). DISCONNECTED lasts only 1–2 ms and is practically invisible in STATUS. An extra transition DISCONNECTED → BACKOFF was noted.
R2 Auto-reconnect / backoff	FAIL	Scenario A: backoff drops from 500 to 50 ms and never grows (retry 2→240 ≈55 ms between attempts). The node did not hang (BUG-01, Major).
R3 Offline buffering	PASS	Scenario B: all 5 messages were buffered (flush_start count=5). For more than 8 messages see R4.
R4 Buffer boundary	FAIL	Scenario C: buffer=12 with capacity 8, dropped=0, IDs 1–4 disappeared without a record. Scenario D (NET DOWN): dropped=0 after 100 offline messages. DROP_OLDEST is announced but not executed (BUG-03).
R5 Recovery	PASS	Scenarios A, B, C: after NET UP the node returns to CONNECTED within 1–14 ms, flushes the buffer and buffer becomes 0. The content of the flush is defective, which is reported under R4 and R6.
R6 Duplicates	FAIL	B: sent=10 for 5 SEND; C: IDs 5–8 ×2, IDs 9–12 ×4; D (NET DOWN): sent +200 and duplicates +100 per run. No duplicates online, so the

		defect is in the flush after reconnect (BUG-02, Major).
R7 Memory stability	FAIL	Scenario D: ≈80 B lost per message, never recovered, −22.9 % over 1,000 messages. Critical by projection (≈4,400 messages); behavior at exhaustion was not tested (BUG-04, Critical).
R8 Evidence	PASS	STATUS contains all required fields: state, buffer/capacity, sent, dropped, duplicates, reconnect_attempts, heap.